What Matters in Layout-Bottlenecked Satellite-to-Street View Synthesis?
A Low-Cost, Reproducible Study with Frozen Foundation Features

Muhammad Anas Ahmed Shaikh[1,*] Hamdan Vohra[1]

[1]FAST National University of Computer and Emerging Sciences, Karachi Campus

[*]Corresponding author: anasahmed81103@gmail.com ORCID: 0009-0004-0449-9248

Abstract

Satellite-to-street view synthesis asks a model to produce a ground-level panorama from an overhead tile. On CVUSA, we study a deliberately simple two-stage pipeline in which a predicted semantic layout is the only scene-structure signal, and we measure which design choices matter. Stage 1, a 1.53M-parameter PolarLayoutNet on frozen DINOv2 features, reaches 0.514 mIoU against SegFormer pseudo-labels, +0.112 over CrossNet retrained on the same labels. Ablations show that DINOv2 instead of VGG-16 (+0.028) and the full tile instead of a 224 px crop (+0.012) matter, while polar versus dense mapping, a ground-plane prior and learned offsets change mIoU by at most 0.001. In Stage 2, adding a geometrically warped aerial image to SPADE conditioning lowers LPIPS with both predicted and oracle layouts (paired 95% bootstrap intervals exclude zero), and swapping oracle for predicted layouts raises clean-FID from 141.0 to 182.1, the largest gap we measure. Our clean-FID remains far behind published methods (24–41 under different protocols); labels are pseudo-labels, building IoU is 0.21, we use one dataset and one training seed, and the adversarial term was largely inactive after early training. Code, logs, checkpoints and the evaluation harness are released.

1 Introduction

Generating a street-level panorama from a satellite tile is a long-standing cross-view problem [1, 3, 4]. The two views share little appearance: the aerial tile shows roofs and canopy from above, while the panorama is dominated by sky, vegetation, buildings and the road ahead. Recent methods close part of this gap with explicit geometry [6, 7] or diffusion models [8].

We instead study a deliberately simple and cheap pipeline whose structure is easy to measure. Stage 1 predicts a four-class panorama layout (sky/other, vegetation, road, building) from the aerial tile. Stage 2 renders a 256×512 panorama from that layout, optionally with a warped copy of the aerial image. Because the layout is the only learned scene-structure signal, it is a bottleneck by design. Swapping the predicted layout for an *oracle layout*, the segmentation of the target panorama, measures how much that bottleneck costs.

Figure 1 shows the full pipeline on one test image. The study makes three contributions.

- **An efficient layout predictor and a map of what matters.** PolarLayoutNet trains 1.53M parameters on frozen DINOv2 features [12] in about four GPU-hours on a 4 GB GPU and reaches 0.514 mIoU. Feature choice and input coverage matter; the polar geometry details we tried do not.

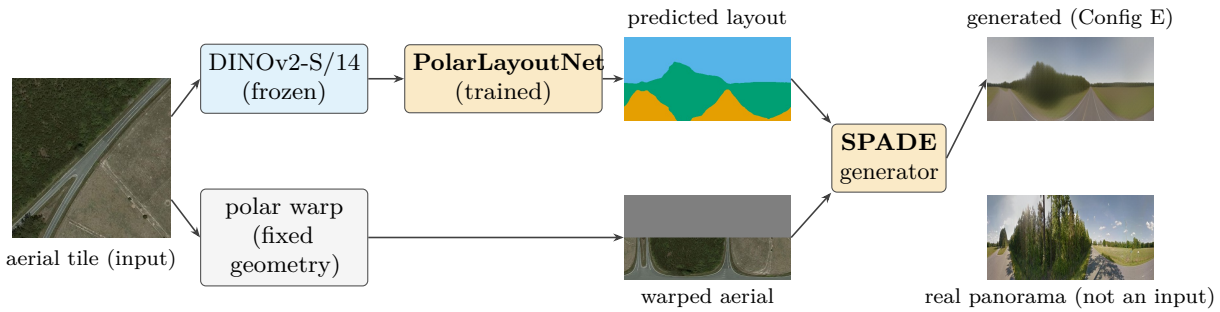


Figure 1: Pipeline on test image 500 (Config E, LPIPS 0.624, the 43rd percentile of the test set). Blue: frozen; orange: trained. Stage 1 encodes the tile with a frozen DINOv2 backbone and a trained PolarLayoutNet; the polar warp has no learned parameters. The SPADE generator (Stage 2) receives the predicted layout and the warped tile's pixels; it is trained with oracle layouts and sees predicted layouts only at test time. Layout colours: sky/other, vegetation, road, building.

- **A controlled Stage 2 ladder with paired uncertainty.** Five configurations separate the generator, the layout source and warped-aerial conditioning; the oracle gap quantifies the bottleneck.
- **Training diagnostics and a full release.** We document a discriminator that saturated from step one and release code, logs, checkpoints and the evaluation harness.

The results do not reach the realism of published methods (Section 7), and we state the confounds of each comparison next to it.

2 Related Work

Cross-view GANs and geometry. Regmi and Borji [3] synthesise aerial-to-street views with conditional GANs that also generate the target-view segmentation. SelectionGAN [4] conditions on an image and a target semantic map and refines a coarse result with multi-channel attention selection. S2SP [6] establishes explicit geometric correspondences and projects the satellite image to the street viewpoint before generation, and Toker et al. [5] train street-view synthesis jointly with cross-view retrieval. Our warped-aerial input is a much simpler, fixed polar resampling in the same spirit.

Density- and diffusion-based methods. Sat2Density [7] learns a density field from satellite-ground pairs without depth supervision, and Sat2Density++ [9] learns a neural radiance field to render street-view panoramas and videos. CrossViewDiff [8] conditions a diffusion model on estimated satellite scene structure and cross-view texture mapping, and its Table 1 collects the CVUSA numbers we quote in Table 6.

Layout prediction. CrossNet [1] predicts the ground-level semantic layout from aerial imagery, supervised by segmentations of co-located ground images, and uses aerial features to hallucinate a plausible panorama. It is our Stage 1 baseline. CVUSA was introduced for cross-view geolocalisation [2].

Foundation features and semantic synthesis. DINOv2 [12] provides all-purpose visual features that work without fine-tuning; register tokens [13] remove high-norm artefacts and give smoother feature maps. Pix2Pix [10] learns image-to-image mappings with a conditional adversarial loss.

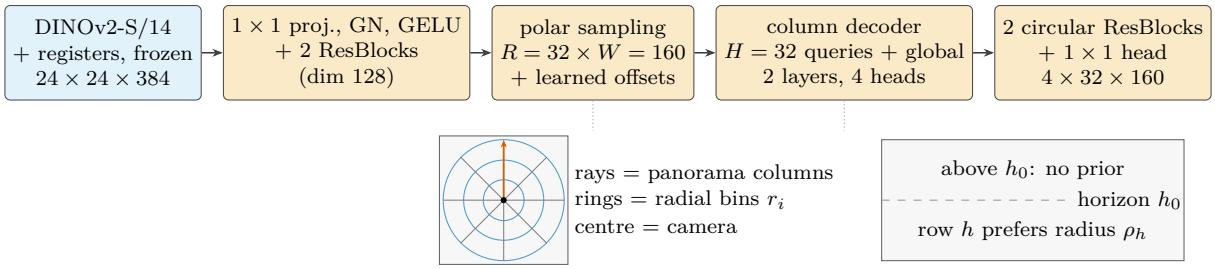


Figure 2: PolarLayoutNet. Frozen DINOv2 tokens of the full tile are projected, resampled along rays from the tile centre (one ray per panorama column), decoded column by column with attention biased by a ground-plane prior, and refined with circular padding so that the panorama wraps around. Only the orange blocks are trained (1.53M parameters).

SPADE [11] injects the semantic layout through spatially-adaptive normalisation so that the layout is not washed out by normalisation layers. Our contribution is not a new generator but a controlled measurement: we keep the encoder frozen, make the predicted layout an explicit bottleneck, quantify its cost with oracle layouts, and report paired intervals for every per-image comparison.

3 Method

3.1 Data, labels and protocol

We use CVUSA [2] with its standard split of 35,532 training and 8,884 test pairs. Each pair holds a 750×750 aerial tile and a 1232×224 street-view panorama whose centre is the tile centre. The released code calls the test split "val"; we call it the test set throughout.

CVUSA provides no street-view semantic labels. We label every panorama with SegFormer-B0 [14] fine-tuned on ADE20K and map its classes to four: sky, vegetation, road and building (the mapping is listed in Section B). The four classes are coarse groups rather than pure categories: class 0 combines sky and water with every ADE20K class we did not assign elsewhere (for example person, pole and signboard), so we call it sky/other; class 1 groups vegetation with natural terrain such as mountain, rock, sand, hill and land; class 2 includes vehicles and bare earth; and class 3 includes other built structures such as fences, railings and bridges, which may partly explain its low IoU. Relabelling 44,416 pairs took 132.2 minutes. These pseudo-labels are the Stage 1 targets and the Stage 2 oracle layouts, and all mIoU numbers use them. For Stage 1 model selection we hold out a 2,000-image minival set from the training split (seed 0) and train on the remaining 33,532.

3.2 Stage 1: PolarLayoutNet

Features. The full 750×750 tile is resized to 336×336 and encoded once with DINOv2-S/14 with registers [12, 13]; the $24 \times 24 \times 384$ patch grid is cached in half precision. A 1×1 projection to 128 channels, GroupNorm, GELU and two residual blocks produce the feature map $\mathbf{F}$ (Figure 2).

Polar sampling. Each panorama column corresponds to an azimuth and each ground pixel to a distance from the camera. We therefore resample $\mathbf{F}$ on $R = 32$ radial bins and $W = 160$ azimuths,

$$r_i = \frac{i + 0.5}{R}, \qquad \theta_j = \frac{2\pi(j + 0.5)}{W} - \pi, \qquad \mathbf{p}_{ij} = (r_i \sin\theta_j,\ -r_i \cos\theta_j), \tag{1}$$

in normalised tile coordinates, with bilinear sampling $\mathbf{P}_{ij} = \mathbf{F}(\mathbf{p}_{ij} + 0.1 \tanh \boldsymbol{\delta}_{ij})$. The offsets $\boldsymbol{\delta}$ come from a zero-initialised convolution over the first sampling pass. A learned radial embedding is added to every sample.

Column decoder. For every azimuth j, $H = 32$ learned row queries attend over that column's R samples plus one global token, a linear map of the mean of $\mathbf{F}$. The decoder has two pre-norm layers with 4 heads and a feed-forward width of twice the model dimension, and is shared across columns. Its attention logits receive an additive bias initialised from a ground-plane prior: with horizon row $h_0 = 0.5H$, a row h below the horizon prefers the radius ρ_h at which the ground would appear,

$$\rho_h = \min\Big(1, \frac{H - (h + 0.5)}{H - h_0}\Big), \qquad b_{hi} = -\frac{1}{2}\Big(\frac{(r_i - \rho_h)R}{1.5}\Big)^2, \tag{2}$$

and rows above the horizon receive $b_{hi} = 0$. The bias stays trainable. Two residual blocks with circular padding along the azimuth and a 1×1 head produce $4 \times 32 \times 160$ logits, which are bilinearly upsampled to 224×1232 for evaluation.

Loss and training. We minimise focal loss [21] plus soft Dice loss (Equation (4) in Section A) on labels at head resolution, train for 8 epochs with AdamW and keep the checkpoint with the best minival mIoU.

3.3 Stage 2: layout-to-panorama generators

Generators. The Pix2Pix U-Net baseline [10] (54.42M parameters) maps the one-hot layout to RGB. It is warm-started from step 197,766 (epoch 38) of an earlier run on a previous label version (from our course project) and fine-tuned for 15 epochs with $\lambda_{L1} = 100$ and no perceptual loss. The SPADE generator [11] (92.09M parameters) starts from the conditioning map downsampled to 8×16, applies seven SPADE residual blocks with five $2\times$ nearest-neighbour upsamplings, and outputs 256×512 RGB; every normalisation layer is modulated by the resized conditioning map (hidden width 128). There is no noise input, so the generator is deterministic.

Warped-aerial conditioning. We resample the aerial RGB tile into panorama geometry with a fixed polar grid: the tile centre maps to the bottom row, the tile edge to the horizon row, azimuth runs clockwise, and north sits at the centre column. The orientation was calibrated once on 400 training pairs by matching warped and panorama road masks. Rows above the horizon are set to zero. The three warped channels are concatenated with the four layout channels (layout resized with nearest neighbour, aerial bilinearly), which adds 0.09M parameters (92.18M in total). Figure 8 shows two warps.

Losses and discriminator. A conditional PatchGAN discriminator D (2.77M parameters) sees the layout concatenated with a real or generated image; it receives the layout only, not the warped aerial. We use least-squares GAN losses [20] with a smoothed real target of 0.9,

$$\mathcal{L}_D = \tfrac{1}{2}\big[(D(x) - 0.9)^2 + D(\hat{x})^2\big], \qquad \mathcal{L}_G = (D(\hat{x}) - 1)^2 + \lambda_{L1}\|\hat{x} - x\|_1 + \lambda_p \sum_l \|\phi_l(\hat{x}) - \phi_l(x)\|_1, \tag{3}$$

where ϕ_l are the relu1_1 to relu4_1 activations of an ImageNet VGG-19 (4 layers) as a perceptual loss [19]. SPADE uses $\lambda_{L1} = \lambda_p = 10$. All Stage 2 models train for 15 epochs (2,220 steps per epoch) with batch 16, Adam ($\beta = (0.5, 0.999)$, learning rate 2×10^{-4}), 3 generator steps per discriminator step and mixed precision. D is randomly initialised in every evaluated configuration (A–E); a warm-started D failed in our diagnostic runs (Section 6). Generators are always trained on oracle layouts.

Table 1: Stage 2 configurations. The oracle layout is the SegFormer segmentation of the target panorama; the predicted layout is the Stage 1 output. Each uses the last checkpoint saved by its training run.

Config	Generator	Layout at test time	Warped aerial	Generator step
A	Pix2Pix U-Net (54.42M)	oracle	–	231,066
B	SPADE (92.09M)	oracle	–	33,300
C	SPADE (92.09M)	predicted	–	33,300
D	SPADE + aerial (92.18M)	oracle	✓	32,000
E	SPADE + aerial (92.18M)	predicted	✓	32,000

3.4 End-to-end inference

At test time Stage 1 logits are bilinearly upsampled to 256×512, converted by argmax to a one-hot layout in $[-1, 1]$, and passed to the generator together with the warped aerial image. Stage 2 never sees a predicted layout in training, so Configs C and E also measure this train-test shift.

4 Experimental Setup

Metrics. Stage 1 reports pixel accuracy, per-class IoU and mIoU over all 8,884 test panoramas at 224×1232. For Stage 2 we compute, per test image at 256×512, SSIM, PSNR, LPIPS with AlexNet features [15] and a *Laplacian sharpness gap* $|\mathrm{Var}(\nabla^2 Y) - \mathrm{Var}(\nabla^2 \hat{Y})|$ on 8-bit luma Y. This gap is our own measure and is not the SD metric of prior work. Set-level clean-FID [17, 16] and KID [18] use all 8,884 generated and real panoramas. An older torchvision-Inception harness, which we call *legacy FID*, appears only in Section D. Per-image SSIM uses scikit-image `structural_similarity` on uint8 RGB after mapping generator outputs from $[-1, 1]$ to eight-bit RGB, with `channel_axis=2`, `data_range=255`, default `win_size=7` and uniform filter weights (`gaussian_weights=False`). PSNR uses scikit-image `peak_signal_noise_ratio` on the same uint8 RGB tensors with `data_range=255`. LPIPS uses the `lpips` package with AlexNet (`net='alex'`), leaving inputs in $[-1, 1]$. clean-FID and KID call `cleanfid` with `mode='clean'` and batch size 32 on folders of 256×512 PNG RGB images written by `final_eval.py`; clean-FID applies its own resizing and normalisation internally [17].

Uncertainty. Each model was trained once, so our intervals capture test-set sampling, not training variance. Per-image metrics get 95% percentile intervals from 1,000 bootstrap resamples of test images (seed 0); for D vs B and E vs C we bootstrap the per-image paired differences. FID and KID have no intervals. We call a difference significant only when its 95% interval excludes zero. Differences quoted in the text are computed from unrounded values, so they can differ by up to 0.001 (0.1 for FID) from differences of the rounded table entries.

Configurations. In Table 1, B and C share one checkpoint, as do D and E, so C−B and E−D isolate the layout source. D−B and E−C isolate warped-aerial conditioning up to the training-run differences noted in Section 5.3. Stage 2 had no separate validation split. Several checkpoints of the same runs were scored on the test split during development (Section D), and the Pix2Pix, SPADE and aerial configurations were compared on the same test images while they were being developed, so test data may have influenced which runs and settings we kept; each configuration reports the last checkpoint its run saved.

Compute. Summed over the logs, Stage 1 used 27.6 GPU-hours, Stage 2 16.3 and the final evaluation 1.70, 45.6 GPU-hours in total; relabelling adds 2.20. Section C gives the per-run

Table 2: Stage 1 on the CVUSA test set (8,884 panoramas), scored against SegFormer pseudo-labels. The original CrossNet checkpoint was trained on its own labels, so its gap mixes label and architecture effects; the retrained CrossNet isolates the architecture. Best per column in bold. "Sky" denotes the sky/other class (Section B).

Method	Trained on	mIoU↑	Acc.↑	Sky	Veg.	Road	Bldg.
CrossNet [1], original checkpoint	original labels	0.299	0.434	0.638	0.026	0.438	0.095
CrossNet, retrained	our pseudo-labels	0.402	0.665	0.633	0.533	0.362	0.080
PolarLayoutNet (ours)	our pseudo-labels	**0.514**	**0.745**	**0.689**	**0.661**	**0.494**	**0.209**

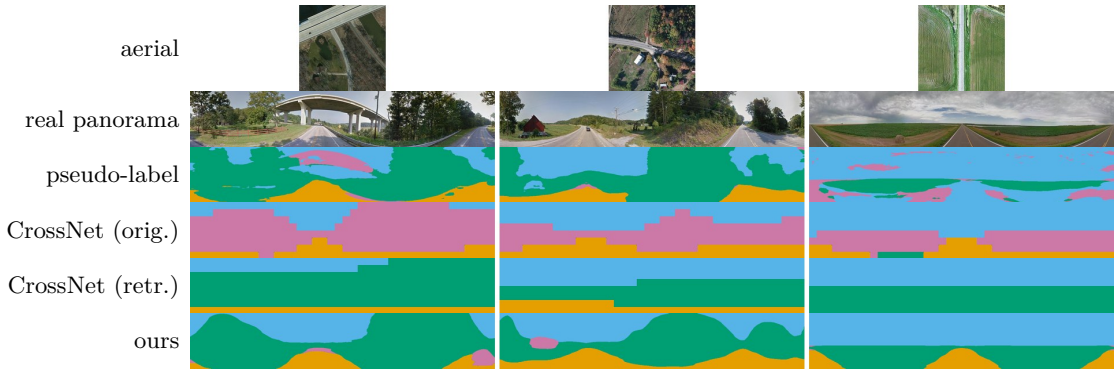


Figure 3: Stage 1 predictions for test images 0, 1 and 2 (the first three test indices; no selection). The CrossNet panels are the predicted-layout panels from each model's evaluation output, recoloured to our palette. Colours as in Figure 1.

breakdown, the GPUs named in the logs and what was not recorded.

5 Results

5.1 Stage 1 layout prediction

PolarLayoutNet reaches 0.514 mIoU and 0.745 pixel accuracy (Table 2). Against CrossNet retrained on the same pseudo-labels, the gain is +0.112 mIoU, with every class improving (road +0.132, building +0.129, vegetation +0.129, sky +0.056). Against the original CrossNet checkpoint the gain is +0.214, but that checkpoint never saw our label definition (it scores 0.454 on its own labels), so this larger number should not be read as an architecture effect. The retrained baseline is, if anything, favoured: its checkpoint was selected on a 1,024-image subset of the test split.

Figure 3 shows the first three test images. The original CrossNet predicts building where the labels have vegetation, matching its vegetation IoU of 0.026, and the retrained CrossNet produces flat, nearly horizontal class boundaries. Our predictions follow the road shape and the vegetation skyline more closely but still miss small buildings, which is consistent with a building IoU of 0.209. **Takeaway.** A small head on frozen DINOv2 features exceeds a retrained CrossNet by +0.112 mIoU on identical labels, at a training cost of about four GPU-hours.

5.2 What matters in Stage 1

Table 3 changes one factor at a time. Two changes matter. Replacing DINOv2 with frozen VGG-16 features costs 0.028 mIoU at a similar parameter count, and encoding a 224 px centre crop instead of the full tile costs 0.012. The crop also coarsens the token grid, so the second effect combines coverage and resolution.

Table 3: Stage 1 single-change ablations on the test set. Δ is the mIoU change from the reference; each row is one run with one seed, so differences of a few thousandths are within run-to-run variation that we did not measure. VGG-16 gives a 21×21 grid and the crop a 16×16 grid; "no prior" starts the bias at zero and "fixed prior" freezes it. Bold marks the best value per column (ties at the displayed precision are all bold). *Also uses mixed precision, batch 32 and a different GPU. †Also uses mixed precision. "Sky" denotes the sky/other class (Section B).

Variant	Params (M)	mIoU↑	Δ	Sky	Veg.	Road	Bldg.
Reference (Section 3.2)	1.53	**0.514**	–	0.689	0.661	0.494	0.209
Input features							
VGG-16 conv5 instead of DINOv2	1.55	0.485	−0.0283	0.664	0.627	0.468	0.181
224 px centre crop instead of full tile	1.53	0.502	−0.0117	0.675	0.643	0.491	0.198
Cross-view mapping and decoder							
dense mapping instead of polar	4.48	**0.514**	+0.0002	0.688	0.660	**0.499**	0.207
no ground-plane prior	1.53	**0.514**	+0.0004	0.690	0.662	0.494	0.209
fixed prior	1.53	0.513	−0.0002	0.688	0.660	0.495	0.210
no learned offsets	1.53	**0.514**	+0.0007	**0.691**	**0.663**	0.497	0.206
Head and loss							
64×320 head*	1.55	0.513	−0.0009	0.684	0.657	0.496	0.214
inverse-frequency weights†	1.53	0.501	−0.0129	0.649	0.623	0.493	**0.237**

Table 4: Stage 2 results on the test set (256×512, 8,884 images). Grey brackets: 95% bootstrap intervals over test images (none for set-level FID and KID). Sharp. gap: our Laplacian sharpness gap. Bold marks the best point estimate per block, not significance (E vs C SSIM is not significant, Table 5).

Config	Generator	SSIM↑	PSNR↑	LPIPS↓	Sharp. gap↓	clean-FID↓	KID↓
Oracle layout							
A	Pix2Pix U-Net	**0.516** [0.513, 0.518]	17.07 [17.02, 17.13]	0.610 [0.608, 0.612]	622 [614, 630]	167.4	0.180
B	SPADE	0.507 [0.504, 0.509]	17.06 [17.01, 17.11]	0.591 [0.588, 0.594]	592 [584, 600]	155.4	0.179
D	SPADE + aerial	0.509 [0.506, 0.511]	**17.28** [17.23, 17.33]	**0.565** [0.562, 0.567]	**586** [578, 594]	**141.0**	**0.163**
Predicted layout (end-to-end)							
C	SPADE	0.458 [0.456, 0.461]	14.65 [14.60, 14.69]	0.671 [0.668, 0.673]	597 [588, 604]	195.0	0.221
E	SPADE + aerial	**0.459** [0.456, 0.461]	**14.82** [14.78, 14.87]	**0.650** [0.647, 0.652]	**592** [584, 600]	**182.1**	**0.209**

The remaining changes form a null set. Dense mapping (a learned token-to-cell matrix with 2.9 times the parameters), the three prior settings, the offsets and a finer 64×320 head all land within 0.001 mIoU of the reference. Inverse-frequency class weights trade 0.013 mIoU for +0.028 building IoU.

Takeaway. In this setting the backbone and the input coverage matter, while the geometric details of the decoder show no measurable effect in single runs; the decoder appears able to learn the cross-view mapping without a hand-built prior.

5.3 Stage 2: the configuration ladder

Table 4 and Figure 4 give the full ladder. With oracle layouts, SPADE (B) improves on the Pix2Pix U-Net (A) by 0.019 LPIPS and 12.0 clean-FID, while A keeps a 0.009 higher SSIM. The two

Table 5: Paired differences for warped-aerial conditioning (with minus without), with 95% bootstrap intervals over per-image differences. The last column is the share of test images whose LPIPS improves. FID and KID differences are set-level and have no intervals. The two runs also differ in learning-rate schedule and resume history (Section 5.3).

Comparison	ΔSSIM	ΔPSNR (dB)	ΔLPIPS	Δclean-FID	ΔKID	LPIPS lower
D−B (oracle)	+0.00196 [+0.00168, +0.00223]	+0.23 [+0.20, +0.25]	−0.026 [-0.027, -0.026]	−14.3	−0.016	87.0%
E−C (predicted)	+0.00019 [-0.00003, +0.00043]	+0.17 [+0.16, +0.19]	−0.021 [-0.021, -0.020]	−12.9	−0.012	85.3%

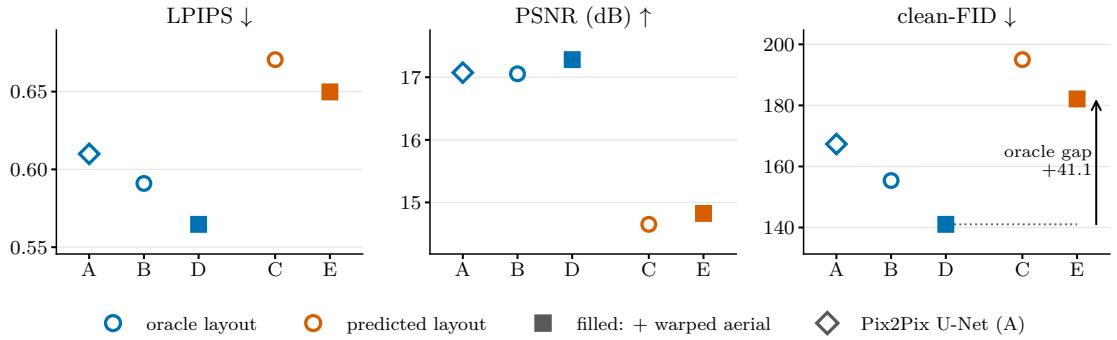


Figure 4: Stage 2 ladder. Points are test-set means; whiskers are 95% bootstrap intervals (often smaller than the marker); clean-FID has no interval. Blue: oracle layout; vermillion: predicted layout; filled squares: with warped aerial. The arrow marks the oracle gap between D and E.

generators differ in architecture, size, λ_{L1}, perceptual loss, learning-rate schedule and initialisation (A was warm-started), so we read this only as evidence that SPADE is a reasonable base.

Adding the warped aerial image is the cleaner comparison (Table 5). With oracle layouts (D−B), LPIPS falls by 0.026 and PSNR rises by 0.23 dB, both intervals excluding zero, and 87.0% of test images improve in LPIPS. With predicted layouts (E−C), LPIPS falls by 0.021 and PSNR rises by 0.17 dB, again excluding zero, and 85.3% of images improve; the SSIM change is not significant. Clean-FID point estimates fall by 14.3 (oracle) and 12.9 (predicted). One confound remains: the layout-only SPADE run decayed its learning rate linearly to zero after epoch 8, whereas the aerial run stayed at 2×10^{-4} in every logged epoch and was resumed from step 12,000. The aerial run therefore lacked the annealing of the layout-only run, which would if anything disadvantage it; the resume history is a second difference we did not isolate.

Takeaway. Warped-aerial conditioning gives a small, consistent LPIPS improvement with both layout sources and lowers clean-FID by 14.3 and 12.9, although the two runs also differ in schedule.

5.4 The oracle gap

The largest effect in the study is the layout source. Moving from oracle to predicted layouts with the same checkpoint raises clean-FID by 41.1 (D to E: 141.0 to 182.1), LPIPS by 0.085 and KID by 0.047, and lowers PSNR by 2.46 dB and SSIM by 0.050. Without the aerial input the gap is similar (39.6 clean-FID, B to C). In clean-FID the oracle gap is 2.9 times the warped-aerial gain, and in LPIPS it is 3.2 times. Part of the gap may come from the train-test shift in Section 3.4, because the generator never saw predicted layouts during training; the two causes are not separated here.

Takeaway. The largest effect we measure is the layout source: a perfect layout would be worth roughly three times what the aerial image adds, although we cannot separate Stage 1 error from the train–test mismatch.

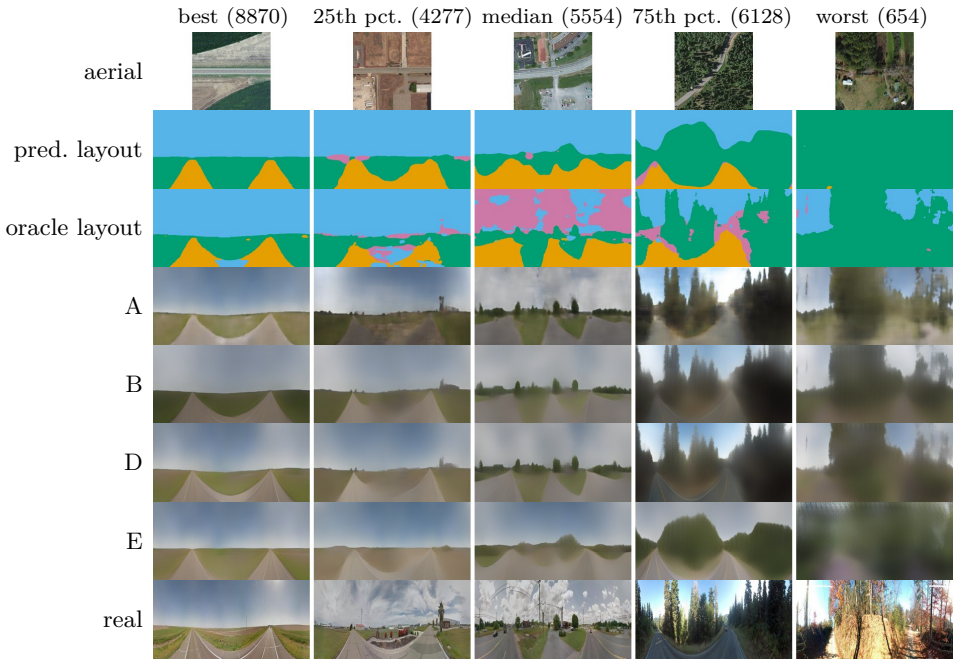


Figure 5: End-to-end grid. Columns are the test images at the best, 25th percentile, median, 75th percentile and worst Config E LPIPS (0.295, 0.569, 0.645, 0.733 and 1.024); test indices in brackets. All panels were regenerated from the released checkpoints with the evaluation code, and their LPIPS matches the stored per-image values. Colours as in Figure 1.

5.5 Qualitative results and failure cases

Figure 5 samples Config E by its own LPIPS quantiles. On open roads (best and 25th percentile) the predicted layout recovers the road shape and the generator produces a plausible, blurry scene. At the median and 75th percentile, the predicted vegetation skyline differs from the oracle and the outputs inherit that error; D shares E's weights but receives the oracle layout. The worst case is a forest view in which both layouts are almost entirely vegetation and every generator returns a uniform blur; Figure 7 shows six uncurated samples; the next three worst cases (test images 2955, 668 and 35) are also dense vegetation scenes with a nearly uniform predicted layout (LPIPS 1.012, 1.010 and 1.001). All samples are far blurrier than the real panoramas.

Takeaway. Visible errors track layout errors, and the low-texture outputs explain why pixel metrics look acceptable while clean-FID stays high.

6 Training Diagnostics

Our first Pix2Pix fine-tuning warm-started both generator and discriminator from the earlier run, and its discriminator was saturated from the first step (Figure 6). An instrumented FP32 run (warm-started weights, a fresh optimiser state and one generator step per discriminator step) shows the mechanism: at step one the discriminator output was 0.899 on real images, almost exactly the smoothed target of 0.9, and 0.000 on generated images. Over 20 diagnostic lines up to step 500 the real mean stayed between 0.876 and 0.910, the fake output never exceeded 0.009 in magnitude, and every logged discriminator loss was below 0.05, the lower edge of the script's healthy band. The mixed-precision run behaved the same way, which rules out mixed-precision underflow as the cause.

A third diagnostic run in FP32 changed two things at once, a randomly initialised discriminator and three generator steps per discriminator step, and behaved as hoped: at step one both outputs

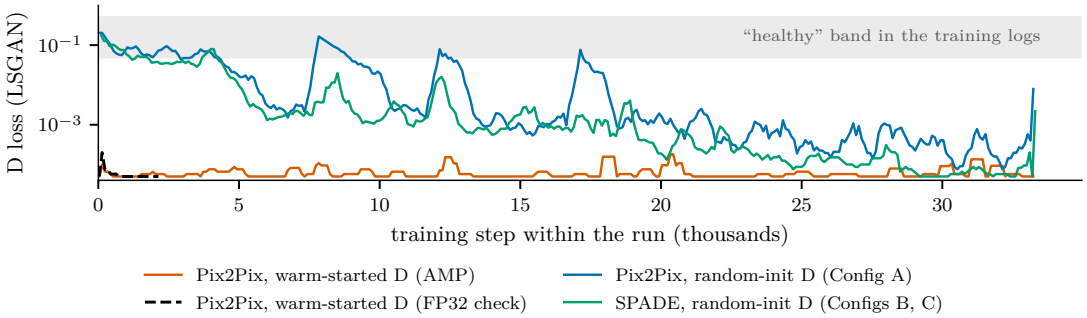


Figure 6: Discriminator loss logged every 100 steps, smoothed with a five-point geometric moving average (log scale). Logged values of 0.0000 are drawn at the floor of 5×10^{-5}. The grey band is the "healthy" range printed by the training script. The warm-started runs sit at the floor from step one; random initialisation starts inside the band but leaves it within the first few thousand steps.

Table 6: Published CVUSA numbers, copied from Table 1 of CrossViewDiff [8] and checked against its PDF. **They are not comparable with ours**: resolution, protocol, FID implementation and labels differ, and we re-ran no published method. The published SD column is omitted (a different measure).

Method	SSIM↑	PSNR↑	FID↓	KID↓
CDTE [5]	0.283	13.82	28.35	0.028
S2SP [6]	0.319	13.69	27.31	0.021
Sat2Density [7]	0.339	14.23	41.43	0.036
CrossViewDiff [8]	0.371	12.00	23.67	0.018
This study, our protocol (clean-FID)				
Config D (oracle layout)	0.509	17.28	141.0	0.163
Config E (end-to-end)	0.459	14.82	182.1	0.209

were close to zero (−0.085 real, −0.083 fake), the epoch-average discriminator loss was 0.0966 and 0.1032 in its two epochs, and only 20% of logged steps fell below 0.05. Because the two changes were made together, we cannot attribute the improvement to initialisation alone.

The full runs with random initialisation started in this range but did not stay there. For Config A (Pix2Pix) the share of logged steps with discriminator loss below 0.05 rose from 27% in the first epoch to 92% from the third epoch on, and the epoch-average loss fell from 0.1220 to 0.0009. For SPADE (Configs B and C) the share rose from 50% to 99% and the average fell from 0.0879 to 0.0001; in the last epoch the adversarial loss averaged 1.00, against weighted L1 and perceptual terms (after multiplying by their weights of 10) of 1.20 and 18.6. The aerial run, logged from epoch 6, was below 0.05 at 99% of logged steps. We read this as a discriminator with little useful gradient left: the adversarial term was largely inactive after the first epochs.

Takeaway. A warm-started discriminator can saturate before training starts. Random initialisation (together with three generator steps per discriminator step) restores the early dynamics but not the late collapse, so the reported generators should be read as mostly reconstruction-trained.

7 Discussion and Limitations

Where the pipeline stands. Our end-to-end clean-FID of 182.1 is far behind the published FID range of 24–41 (Table 6), even allowing for protocol differences. Our higher SSIM and PSNR do not indicate better synthesis: these metrics reward smooth outputs, and our samples are visibly blurrier (Figure 5).

Limitations. All labels are SegFormer pseudo-labels, so mIoU measures agreement with another model rather than with human annotation, and building IoU is only 0.21; there is no human evaluation of the synthesised panoramas. We use one dataset, CVUSA, which covers the United States, and make no claim about other datasets or regions. The 1232×224 panoramas are resized to 256×512 (height $\times$ width), which changes their aspect ratio. Each configuration was trained once, so our intervals cover test-image sampling only. The A-to-B and warped-aerial comparisons carry the confounds stated in Section 5.3, the adversarial term was largely inactive after the first epochs, and Stage 2 was trained only on oracle layouts. We also did not train a generator conditioned on the warped aerial tile alone, so we cannot say whether the layout stage helps relative to direct synthesis.

Future work. The oracle gap points first to a better Stage 1 and to training Stage 2 on predicted layouts, which would also separate layout error from train–test mismatch. A generator conditioned only on the warped aerial tile would test whether the layout stage helps at all. A diffusion refiner with ControlNet-style layout conditioning [22] could add the missing texture, and a second dataset and several training seeds would test how far the findings generalise.

8 Conclusion

A 1.53M-parameter layout head on frozen DINOv2 features reaches 0.514 mIoU on CVUSA pseudo-labels, +0.112 over a retrained CrossNet; the backbone and input coverage matter, the polar mapping details do not. A warped aerial image lowers LPIPS with both layout sources, but the step from oracle to predicted layouts costs 41.1 clean-FID, the largest gap we measure, which points to Stage 1 quality, train–test mismatch, or both as the first things to improve in this design. The released code, logs and checkpoints give a cheap, fully measured baseline against which better layout prediction can be tested.

Reproducibility and Data Statement

Code, configuration files, training logs, the Stage 1 and Stage 2 checkpoints and the evaluation harness are released at `https://github.com/anasahmed81103/AerialToGround` (release v1.0.0) [TODO: confirm the release and the Zenodo DOI are public before submission]. Every number in this paper was checked against the run artefacts by a script shipped with the paper source (`paper/tools.py`). CVUSA was obtained from the authors' shared Dropbox folder (`cvusa_sharing`), following a data-access request to Nathan Jacobs (Washington University in St. Louis), is used for research under its terms, and is not redistributed by us; the figures show 17 distinct test images, identified in their captions.

Author Contributions

A.A. and H.V. jointly developed the original CrossNet and Pix2Pix baselines in a shared course project. A.A. designed and implemented Stages 1 and 2, ran all experiments and analyses, and wrote the paper [TODO: confirm contributions with H.V.].

Acknowledgements

This work began as a Computer Vision course project at FAST-NUCES; the original CrossNet checkpoint and the Pix2Pix initialisation of Config A come from that project. This research received no external funding; cloud GPU time was paid for personally by the first author. AI assistants (Cursor and Claude) were used to help write analysis code and to draft and edit the text; the authors checked all results and claims against the run artefacts and take responsibility for the content [TODO: confirm venue policy].

A Hyperparameters

With softmax probabilities q and one-hot targets y over N pixels and C classes, the Stage 1 loss is

$$\mathcal{L}_{\text{seg}} = -\frac{1}{N}\sum_{n}(1-q_{n,y_n})^{\gamma}\log q_{n,y_n} + \frac{1}{C}\sum_{c}\Big(1-\frac{2\sum_n q_{nc}y_{nc}+1}{\sum_n q_{nc}+\sum_n y_{nc}+1}\Big), \qquad \gamma = 2, \tag{4}$$

computed on labels downsampled to the 32×160 head resolution. Training uses horizontal flips of tile and panorama and the settings in Table 7.

Table 7: Training hyperparameters, read from the run configurations and logs.

	Stage 1 (PolarLayoutNet)	Stage 2 (SPADE; Pix2Pix where different)
Input	DINOv2-S/14 tokens, $24 \times 24 \times 384$	one-hot layout (+ warped aerial), 256×512
Output	$4 \times 32 \times 160$ logits	$3 \times 256 \times 512$ RGB
Trainable parameters	1.53M	92.09M / 92.18M (Pix2Pix 54.42M); D 2.77M
Optimiser	AdamW, lr 3×10^{-4}, weight decay 0.01	Adam, lr 2×10^{-4}, $\beta = (0.5, 0.999)$
Schedule	500 warm-up steps, cosine	Layout SPADE: 2×10^{-4} epochs 1–8, linear decay to 0 by epoch 15 (33,300 steps). SPADE + aerial: constant 2×10^{-4} in logged epochs 6–15, resumed at step 12,000; checkpoint at 32,000.
Batch, epochs	16, 8	16, 15 (2,220 steps per epoch)
Loss	focal ($\gamma = 2$) + Dice	LSGAN + 10 L1 + 10 VGG-19 (Pix2Pix: 100 L1)
Other	grad. clip 1.0, flip augmentation	real label 0.9, 3 G steps per D step, mixed precision
Selection	best minival mIoU (2,000 train images)	last saved checkpoint

B ADE20K to four-class label map

Relabelling follows `scripts/relabel_cvusa.py`: SegFormer-B0 predicts ADE20K indices 0–149, which are remapped to CrossNet-style ids 0 sky/other, 1 vegetation, 2 road and 3 building. Any ADE20K index not listed below maps to sky/other (0).

C Compute

Stage 1 trains on the cached features; the main run took 234.1 minutes on a Quadro P1000 with 4 GB of memory and peaked at 1.98 GB. Stage 2 runs used an RTX 4090 or an RTX PRO 4500

Table 8: ADE20K → four-class mapping (same as `_ADE_TO4` in `relabel_cvusa.py`).

Target	ADE20K id	ADE20K class name (`id2label` of `nvidia/segformer-b0-finetuned-ade-512-512`)
sky/other (0)	2	sky
	21	water
	26	sea
	60	river
	113	waterfall
	128	lake
	All other ADE20K indices (default in script)	
vegetation (1)	4	tree
	9	grass
	16	mountain
	17	plant
	29	field
	34	rock
	46	sand
	66	flower
	68	hill
	72	palm
	94	land
road (2)	6	road
	11	sidewalk
	13	earth
	20	car
	52	path
	54	runway
	80	bus
	83	truck
	91	dirt track
	102	van
building (3)	0	wall
	1	building
	8	windowpane
	14	door
	25	house
	32	fence
	38	railing
	42	column
	48	skyscraper
	61	bridge
	79	hovel
	84	tower
	86	awning

Blackwell, as stated in each log. The full evaluation session, including clean-FID, KID and bootstrap, took about 6 hours (author-reported, not logged).

Not recorded: the training time of both CrossNet baselines, the DINOv2 caching time, the clean-FID/KID time, and epochs 1–5 of the SPADE + aerial run, whose first log segment ended before any epoch completed.

Table 9: GPU-hours as logged. Stage 1 times come from each run's results file and Stage 2 times from the sum of logged epoch times. GPU names come from the logs, except the rows marked local, the class-weighted row and the final-evaluation row, which are author-reported.

Run	GPU	GPU-hours
Stage 1 reference (polar, full tile)	Quadro P1000 (4 GB)	3.90
Stage 1 ablations: no prior / fixed prior / no offsets	Quadro P1000 (local)	3.91 / 3.87 / 3.70
Stage 1 ablations: dense / crop / VGG-16	Quadro P1000 (local)	4.03 / 3.71 / 3.84
Stage 1 64×320 head	RTX 4090	0.45
Stage 1 class-weighted	RTX 4090	0.16
Stage 1 total		*27.6*
Pix2Pix, warm-started D (mixed precision; diagnostic)	RTX 4090	0.08
Pix2Pix, warm-started D (FP32; diagnostic)	RTX 4090	0.07
Pix2Pix, random D, FP32 (diagnostic)	RTX PRO 4500 Blackwell	0.39
Pix2Pix, warm-started D (full run)	RTX 4090	1.12
Pix2Pix, random D (Config A)	RTX PRO 4500 Blackwell	1.73
SPADE (Configs B, C)	RTX 4090	7.81
SPADE + aerial (Configs D, E), epochs 6–15 only	RTX 4090	5.14
Stage 2 total (logged)		*16.3*
Final evaluation, Configs A–E (excluding clean-FID/KID)	RTX 4090	1.70
SegFormer relabelling	not recorded	2.20
Total logged, including relabelling		*47.8*

D Legacy FID

Before the final evaluation we scored checkpoints with an older harness built on the torchvision Inception network (*legacy FID*); its numbers are not comparable with clean-FID. Legacy FID was 164.4 for Pix2Pix with a warm-started D, 157.6 for Config A (160.6 at step 228,846), 129.7 for Config B and 116.2 for Config D. During development the SPADE run was scored at steps 31,080, 32,000 and 33,300, and the Config A run at steps 228,846 and 231,066, all on the test split. The reported checkpoints are the last saved ones, but these evaluations may have influenced later decisions.

E Additional figures

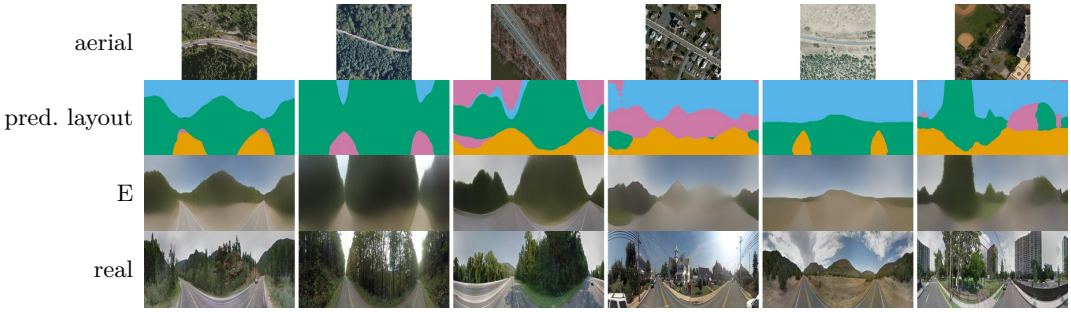


Figure 7: Uncurated samples: six test images drawn with a fixed random seed (test indices 526, 6477, 6539, 6735, 6562 and 1578; Config E LPIPS 0.663, 0.769, 0.570, 0.712, 0.685 and 0.780), shown without selection. Panels were regenerated from the released checkpoints with the evaluation code, and their LPIPS matches the stored per-image values. Colours as in Figure 1.

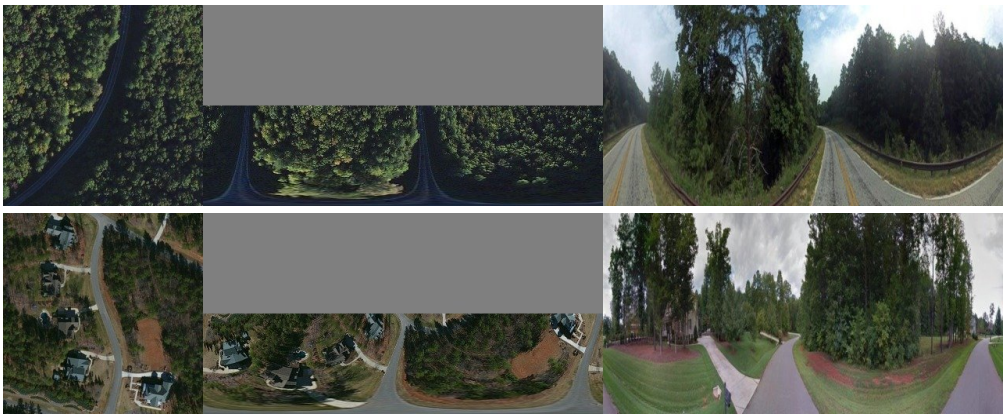


Figure 8: Warped-aerial check for test images 1269 and 3807, from the warp verification output of the released code: the aerial tile, the warped aerial panorama and the real panorama for visual alignment.

References

[1] M. Zhai, Z. Bessinger, S. Workman, and N. Jacobs. Predicting Ground-Level Scene Layout from Aerial Imagery. In *CVPR*, 2017. arXiv:1612.02709.

[2] S. Workman, R. Souvenir, and N. Jacobs. Wide-Area Image Geolocalization with Aerial Reference Imagery. In *ICCV*, 2015. arXiv:1510.03743.

[3] K. Regmi and A. Borji. Cross-View Image Synthesis using Conditional GANs. In *CVPR*, 2018. arXiv:1803.03396.

[4] H. Tang, D. Xu, N. Sebe, Y. Wang, J. J. Corso, and Y. Yan. Multi-Channel Attention Selection GAN with Cascaded Semantic Guidance for Cross-View Image Translation. In *CVPR*, 2019. arXiv:1904.06807.

[5] A. Toker, Q. Zhou, M. Maximov, and L. Leal-Taixé. Coming Down to Earth: Satellite-to-Street View Synthesis for Geo-Localization. In *CVPR*, 2021. arXiv:2103.06818.

[6] Y. Shi, D. Campbell, X. Yu, and H. Li. Geometry-Guided Street-View Panorama Synthesis from Satellite Imagery. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022. arXiv:2103.01623.

[7] M. Qian, J. Xiong, G.-S. Xia, and N. Xue. Sat2Density: Faithful Density Learning from Satellite-Ground Image Pairs. In *ICCV*, 2023. arXiv:2303.14672.

[8] W. Li, J. He, J. Ye, et al. CrossViewDiff: A Cross-View Diffusion Model for Satellite-to-Street View Synthesis. arXiv preprint, 2024. arXiv:2408.14765.

[9] M. Qian, B. Tan, Q. Wang, et al. Seeing through Satellite Images at Street Views. arXiv preprint, 2025. arXiv:2505.17001.

[10] P. Isola, J.-Y. Zhu, T. Zhou, and A. A. Efros. Image-to-Image Translation with Conditional Adversarial Networks. In *CVPR*, 2017. arXiv:1611.07004.

[11] T. Park, M.-Y. Liu, T.-C. Wang, and J.-Y. Zhu. Semantic Image Synthesis with Spatially-Adaptive Normalization. In *CVPR*, 2019. arXiv:1903.07291.

[12] M. Oquab, T. Darcet, T. Moutakanni, et al. DINOv2: Learning Robust Visual Features without Supervision. arXiv preprint, 2023. arXiv:2304.07193.

[13] T. Darcet, M. Oquab, J. Mairal, and P. Bojanowski. Vision Transformers Need Registers. arXiv preprint, 2023. arXiv:2309.16588.

[14] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo. SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers. In *NeurIPS*, 2021. arXiv:2105.15203.

[15] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang. The Unreasonable Effectiveness of Deep Features as a Perceptual Metric. In *CVPR*, 2018. arXiv:1801.03924.

[16] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter. GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium. In *NeurIPS*, 2017. arXiv:1706.08500.

[17] G. Parmar, R. Zhang, and J.-Y. Zhu. On Aliased Resizing and Surprising Subtleties in GAN Evaluation. In *CVPR*, 2022. arXiv:2104.11222.

[18] M. Bińkowski, D. J. Sutherland, M. Arbel, and A. Gretton. Demystifying MMD GANs. In *ICLR*, 2018. arXiv:1801.01401.

[19] J. Johnson, A. Alahi, and L. Fei-Fei. Perceptual Losses for Real-Time Style Transfer and Super-Resolution. In *ECCV*, 2016. arXiv:1603.08155.

[20] X. Mao, Q. Li, H. Xie, R. Y. K. Lau, Z. Wang, and S. P. Smolley. Least Squares Generative Adversarial Networks. In *ICCV*, 2017. arXiv:1611.04076.

[21] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal Loss for Dense Object Detection. In *ICCV*, 2017. arXiv:1708.02002.

[22] L. Zhang, A. Rao, and M. Agrawala. Adding Conditional Control to Text-to-Image Diffusion Models. In *ICCV*, 2023. arXiv:2302.05543.